



UD3

INTRODUCCIÓN A TYPESCRIPT

3.8 TYPESCRIPT

MP_0373

Lenguaje de marcas y
Sistemas de gestión de
la información

Introducción

JavaScript, como lenguaje, se enfrenta a algunas limitaciones importantes que se han ido solucionando con las nuevas versiones del estándar web.

Algunas de ellas como el tipado débil de las variables, el uso de estructuras de datos más complejas o el aviso de errores en tiempo de desarrollo no de ejecución son algunos puntos que JavaScript hoy en día no ha conseguido solventar y que TypeScript nos ayuda solventarlo

El hecho de que estudiemos TypeScript en este curso es porque angular lo usa como piedra fundamental del desarrollo de sus componentes, así que este tema intenta servir como punto de partida y toma de contacto para aprender a trabajar con TypeScript.

Con TypeScript nos enfrentamos a un conjunto de herramientas creadas por encima de JavaScript.

```
ng serve
```

El objetivo fundamental de este tema es aprender los conceptos más importantes de TypeScript y como se aplica para usarlo como herramienta de desarrollo en nuestras aplicaciones de angular.

Instalación y uso de TypeScript

Aunque TypeScript en Angular no necesita de ninguna instalación es cierto que TypeScript se puede usar no solo en Angular si no también en solitario para el desarrollo con JavaScript o en otros frameworks como React, Nest JS, etc.

Aprender TypeScript no es muy difícil si ya sabes JavaScript y te aporta herramientas muy interesantes.

La manera más fácil de usar TypeScript de forma nativa es instalarlo por defecto dentro de nuestra máquina de trabajo

```
npm install -g typescript
```

Recuerda que la bandera `-g` significa que no instalamos TypeScript dentro de una carpeta concreta si no de forma global en nuestro equipo.

Para estar seguros de que TypeScript se ha instalado correctamente tenemos que aplicar el siguiente comando en nuestra terminal. La respuesta esperada es la versión de TypeScript actual que acabamos de instalar.

```
tsc --version
```

Puntos a tener en cuenta cuando se usa TypeScript

Los ficheros con los que vamos a trabajar tendrán extensión `.ts` no extensión `.js` como estábamos usando hasta ahora.

Para poder trabajar con ellos debemos compilarlos y transformarlos a JavaScript con el siguiente comando.

```
tsc nombre_fichero.ts
```

Tipado

Una de las mejoras que trae TypeScript respecto a JavaScript es el tipado fuerte de variables. Este es opcional pero muy recomendable por parte de los desarrolladores ya que nos ayuda a cometer menos errores a la hora de programar.

Por qué es tan importante el tipado para los desarrolladores:

Mediante el uso de los diferentes tipos a la hora de definir nuestras variables, limitamos su uso. Las variables que creemos solo vamos a poder utilizarlas para almacenar valores de esos tipos en concreto.

Para especificar el tipo de unas variables podemos hacerlo con la siguiente estructura.

```
let nombreVariable: tipo = valor  
let nombre: string = "Juan"
```

Diferentes tipos de datos en TypeScript y cómo se usan

Boolean

Se trata del tipo más básico, en el que especificamos si el valor es verdadero (true) o falso (false).

```
let termina: boolean = false
```

Number

Es la definición genérica que usamos para hacer referencia a cualquier tipo de número en TypeScript.

Aparte de eso, el tipo number también soporta el uso de binarios, octales y hexadecimales.

```
let decimal: number = 23
let decimal2: number = 12.3
let hex: number = 0xf00d
let binario: number = 0b1001
let octal: number = 0o744
```

String

Utilizamos el tipo string para hacer referencia a todas las cadenas de tipo literal. Podemos limitar estas cadenas con comillas dobles ("), simples (') o podemos utilizar los template strings típicos de JavaScript (`).

```
let nombre: string = "Roberto"
let apellidos: string = 'García'
let saludo: string = `Hola, me llamo ${nombre} ${apellidos}`
```

Array

Del mismo modo que pasa en JavaScript, los arrays nos permiten almacenar varios valores dentro de una estructura.

La diferencia aquí es que vamos a especificar el tipo de los valores almacenados en nuestro array.

Se puede especificar de dos maneras:

```
let list: number[] = [1,2,3,4,5]
let list2: Array<number> = [6,7,8,9]
```

Tupla

Este tipo de dato nos permite representar un array que, normalmente dispone un tamaño fijo y en el que limitamos a dos únicos tipos los datos que vamos a poder incluir dentro del mismo. Lo vemos con un ejemplo:

```
let t: [string, number]
t = ['hola', 32]
// Únicamente recibe string y number
```

Se podrían incluir más elementos de los tipos especificados, pero se suele limitar su uso a pares de valores.

Enum

Utilizamos este tipo de dato específico para aplicar un nombre más amigable a un conjunto de datos numéricos.

```
enum Size {Small, Medium, Big}
let s: Size = Size.Medium
```

Por defecto, los elementos de tipo enum comienzan a definir sus valores a partir de 0, pero podemos cambiar ese comportamiento, modificando alguno de los valores. Vemos cómo modificar el valor de inicio por defecto.

```
enum Size {Small=1, Medium, Big}
let s: Size = Size.Medium
console.log(s) // 2
```

Incluso podríamos definir todos los valores.

```
enum Size {Small=3, Medium=6, Big=9}
let s: Size = Size.Medium
console.log(s) // 6
```

Podemos recuperar también el nombre relacionado con alguno de los valores:

```
enum Size {Small=3, Medium=6, Big=9}
let nbColor: string = Size[6]
console.log(nbColor) // Medium
```

Any

Utilizamos este tipo para definir aquellas variables de las cuales desconocemos su tipo. Este tipo de variables suelen venir del análisis de contenido dinámico o de librerías de terceros que desconocemos.

```
let duda: any = 4
duda = "almacenamos un string"
duda = true
console.log(duda) // true
```

Funciones

En typescript las funciones que reciben o que devuelven parámetros también pueden tipar los datos tanto en un sitio como en otro.

```
function sumar(pNumeroA: number, pNumeroB: number): number {
    let resultado: number = pNumeroA + pNumeroB;
    return resultado;
}
```

Cuando una función no devuelve nada usamos el tipo de dato contrario a **ANY** que es **VOID**.

```
function saludo(): void{
    console.log("Hola Mundo!!")
}
```

Muchas veces necesitamos trabajar de manera concreta con variables de las cuales desconocemos su tipo.

Para poder decirle al compilador que «confíe en nosotros» y trate a las variables con el tipo que necesitamos, podemos hacerlo de dos maneras diferentes.

```
let valor: any = "Es una cadena"  
let sizeValor: number = (<string>valor).length  
let sizeValor2: number = (valor as string).length
```

Interfaces

Técnicamente un interfaz es un mecanismo de la programación orientada a objetos que trata de suplir la carencia de la herencia múltiple. Casi todos los lenguajes de programación que implementan la programación orientada a objetos no ofrecen la posibilidad de definir una clase que extienda de varias clases a la vez y muchas veces sería lo deseable. Y es aquí donde los interfaces cobran su uso.

Mediante las interfaces de TypeScript podemos definir contratos que, las clases que implementen dichos interfaces tendrán que cumplir.

De esta manera nos aseguramos de que las clases que implementan ciertos interfaces van a tener los mismos métodos y propiedades, aparte de las suyas propias.

```
interface IPersona{  
    name: string  
    getName(): string  
}  
  
class Persona implements IPersona{  
    name: string  
    getName(): string{  
        return this.name  
    }  
}
```

Si en el ejemplo anterior, la clase Persona no cumple con alguno de los nombres o tipos de las propiedades o métodos definidos en el interfaz, obtendremos un error de Typescript indicándonos que algo va mal en nuestro desarrollo.

Como has podido ver los interfaces no son difíciles de definir y usar. En el mundo de la programación orientada a objetos se usan bastante y para nosotros suponen una ayuda a la hora de definir que métodos o que propiedades va a tener nuestra clase.

El uso de interfaces nos obliga a mantener buenas prácticas de programación y a cambio nos provee de una serie de mecanismos que nos permite detectar errores de una forma más fácil.

Decoradores

Un decorador es una función que, dependiendo de que cosa queramos decorar, sus argumentos serán diferentes.

Usan la forma `@expression` donde «expression» evaluará la función que será llamada. A continuación, explicaré los decoradores más frecuentes:

Decoradores de clase

Es aplicado al constructor de la clase y puede ser usado para observar, modificar o reemplazar la definición inicial de la clase. Su único argumento es `target` que vendría siendo la clase decorada, tipado como `Function` o `any`:

```
function Bienvenida(target: Function): void{
    target.prototype.saludo = function(): void{
        console.log("Hola Mundo")
    }
}
```

```
@Bienvenida
class Saludar{
    constructor(){}
}
```

```
let s = new Saludar()
s.saludo()
```

Si quisiéramos modificarlo, agregando un parámetro en la llamada al decorador, podríamos hacerlo de la siguiente manera

```
function Bienvenida(saludo: string){
  return function(target: Function){
    target.prototype.saludo = function(): void{
      console.log(saludo)
    }
  }
}

@Bienvenida("Saludando")
class Saludar{
  constructor(){}
}

let s = new Saludar()
s.saludo()
```

Podemos decorar de la misma manera, propiedades definidas en nuestras clases. Primero definimos la función decoradora.

```
function LogCambios (target: Object, key: string){
  var valueProp: string = this[key]
  if(delete this[key]){
    Object.defineProperty(target, key, {
      get: function(){
        return valueProp
      },
      set: function(newValue){
        valueProp = newValue
        console.log(`${key} es ahora ${valueProp}`)
      }
    })
  }
}
```